

기존설계 특징점 학습 — 구현 현황 분석 및 개발계획

대상 사양: `routing3d_existing_design_feature_learning_process.md`
분석 기준: 현재 repo(CLAUDE.md §4-§8 + `csharp/Routing3D.Viewer/` + `python_experiments/routing3d_py/` + `db/schema/`)
작성: 2026-06-14

0. 요약 (한눈에)

사양 문서가 정의한 2 단 구조(① 특징점 학습 → ② 특징점 기반 자동설계) 중, 자동설계 적용(②)은 대부분 구현되어 있고, 특징점을 정식 저장소로 분리·집계하는 학습 계층(①)은 부분 구현, 장애물-배관 연관 학습은 전무하다.

영역	사양	현 구현	비고
데이터 로드(§3.1)	☑	☑	<code>ObstacleDbLoader</code> (장애물/장비/덕트/레터럴/공간/route_path/segment)
경로-작업 매칭(§3.3)	☑	☑	<code>FindMatchingExistingPipe</code> (forward/reverse 비용)
PoC 접속면 학습(§4.1-5.3)	☑	☑	<code>pattern_learn</code> + <code>route_stub_template</code> 뷰+ <code>PatternStore</code> voting
스텝 학습·라우팅(§4.2-5.4-7.3)	☑	☑	<code>StubExtractor</code> / <code>route_stub_pattern</code> /스텝 우선 라우팅
Rack z 학습·주입(§4.3-5.5-7.4)	☑	☑	<code>learn_rack_levels</code> / <code>BuildRackLevels</code> / <code>BuildLearnedTiers</code> / <code>rack_levels</code>
번들/회랑 학습·주입(§4.4-5.6-7.5)	⊙	⊙	<code>bundle_detect</code> + <code>BundleStore</code> + <code>w_corridor</code> — trunk_centerline/branch 미저장

영역	사양	현 구현	비고
corridor attraction(\$5.7)	☑	☑	w_corridor+r3d_set_corridor_cells
관경/간격(\$4.6·7.6)	⊙	⊙	per-task radius+r3d_set_pipe_gap+min_straight — 부속(밸브/리듀서) 학습 없음
다중 순서·rip-up(\$5.9·7.8)	⊙	⊙	diameter/longest/utility+rip-up+CBS — 합성 priority(혼잡도/중심성) 없음
경로 형상 특징 저장(\$4.3)	⊙	⊙	AutoDesignReport/DbRouteDiag 가 즉석 계산 — 저장 테이블 없음
유사도 평가(\$5.10·9)	⊙	⊙	grouping factor 4 성분만 — 접속면/회랑겹침/per-task 7 성분 없음
장애물-배관 연관 학습(\$4.5·5.8·7.7)	✕	✕	전무 — 분류/우회 profile/soft-clearance/preferred-bypass 모두 없음
통합 feature 저장소 route_feature_*(§6.2)	✕	✕	전무 — 현재 route_stub_pattern+route_bundle_group 로 분산
group_profile 머티리얼라이즈(\$6.2.6)	✕	✕	런타임 뷰 조회로 대체 중
운영 학습 파이프라인(\$8)	⊙	⊙	batch --write-db 있음 — 증분/hash/품질게이트 없음
모듈 분리(\$11 아키텍처)	⊙	⊙	로직이 SceneViewModel/DbRouteDiag/pattern_learn 에 산재
feature 시각 레이어(\$9.3)	⊙	⊙	점유/방문/충돌/기존/자동 토글 있음 — rack/corridor/stub/face overlay 부분

대략적 구현율: 자동설계 적용(②) ≈ 75%, 특징점 학습 저장소(①) ≈ 35%, 장애물-배관 연관(횡단) ≈ 0%.

사양은 OpenVDB 점유맵을 전제하나 현 엔진은 Dense/Sparse/**Implicit**(5M 셀 초과 자동전환)로 동일 목적을 달성하고 OpenVDB capi 는 보류 상태다(기능 동치, 문서상 용어 차이일 뿐 겹아님).

1. 이미 구현된 것 (재사용 자산)

신규 개발 시 다시 만들지 말고 확장해야 할 기존 자산:

사양 항목	구현 위치
DB 로더(장애물·장비·덕트·레터럴·공간·route_path·segment_detail)	ObstacleDbLoader.cs
경로-작업 매칭(방향 판정)	SceneViewModel.FindMatchingExistingPipe
스텝 추출(런압축→지터흡수→엘보)	StubExtractor.cs · pattern_learn._walk_stub
접속면 학습/저장	pattern_learn.py·pattern_db.py·route_stub_patterns.s
접속면 조회/투영	PatternStore.cs·SceneViewModel.{LearnedFace,Lif
rack z 학습/주입	pattern_learn.learn_rack_levels·SceneViewModel.
번들 탐지/저장/주입	bundle_detect.py·route_bundle_group.sql·BundleStor
corridor attraction	엔진 w_corridor·C ABI r3d_set_corridor_cells·Buil
관경 물리	C ABI r3d_set_per_task_radius/r3d_set_pipe_gap/n
다중 라우팅·회복	엔진 route_multi/route_ripup/CBS(r3d_set_cbs_dep
기존설계 추종 복제	SceneViewModel.{ReplicateMatchedPipes,Replicate
실패 진단	엔진 RouteFail enum→ExplainFailure

사양 항목	구현 위치
비교 리포트(그룹핑 4 성분·스냅샷·CSV/HTML/PDF)	AutoDesignReport.cs
헤드리스 A/B 진단	DbRouteDiag.cs(--dbroute)

2. 핵심 갭 (개발 대상)

G1. 장애물-배관 연관 학습 (§4.5·§5.8·§7.7) — 최우선·전무

사람 설계의 "기둥/H-beam 앞에서 미리 꺾고 충분히 띄우고 rack 으로 복귀"를 재현하는 핵심인데 **하나도 없다**.

- 장애물 유형 분류(COLUMN/H_BEAM/WALL/FRAME/EQUIPMENT/DUCT/LATERAL)와 주축(obstacle_axis) 미산출.
- 장애물-segment 최근접·확장 AABB 교차·전후 bend 거리·우회 side 추출 미구현.
- `relation_score`(5 성분) 인과 판정 미구현.
- 엔진에 **hard zone / soft clearance zone / preferred bypass corridor** 3 계층 비용이 없다(현재는 단일 hard 팽창 + clearance penalty 뿐).

G2. 통합 feature 저장소 `route_feature_*` (§6.2) — 전무

현재 학습 산출물이 `route_stub_pattern`(스텝) + `route_bundle_group`(번들) + 공식 `TB_ROUTE_SEGMENT_TEMPLATE/TB_ROUTE_DESIGN_GROUP` 로 분산. 사양이 제안한 6 개 테이블 미존재:

`route_feature_path·route_feature_anchor·route_feature_stub_template·route_feature_bundle_template·route_feature_obstacle_relation·route_feature_group_profile`.

G3. group profile resolver + 머티리얼라이즈 (§5(fallback)·§6.2.6·§7.1) — 부분/전무

fallback 계층(장비+유틸 → 그룹 → 유틸그룹 → kind)을 **부분적으로** PatternStore 가 하나, 통합 profile 한 곳에서 rack/corridor/face/pitch/radius/priority 를 한꺼번에 주는 `RouteProfileResolver` 가 없다. 매 라우팅마다 개별 뷰를 따로 조회.

G4. 경로 형상 특징 영속화 (§4.3) — 부분(즉석계산만)

detour_ratio-z_histogram-main_rack_z-dominant_axis-complexity 등이
AutoDesignReport/DbRouteDiag 에서 **그때그때 계산**되고 저장되지 않아 재사용·집계 불가.

G5. 유사도 평가 7 성분 + per-task 비교 (§5.10-§9.1) — 부분

현 grouping factor 는 **자동설계끼리의 다발성 4 성분**(랙/밀집/pitch/lane)일 뿐, **기존설계 대비**
접속면 일치·회랑 겹침·rack z 일치·bend/length 차이를 합친 7 성분 similarity 와 작업별
face_match/corridor_overlap/collision_count 리포트가 없다.

G6. 운영 학습 파이프라인: 증분·품질게이트 (§8) — 부분

batch 학습(pattern_learn --write-db)은 있으나 ① route_hash 기반 증분 학습 ②
is_training_usable 품질 게이트(이상 경로 배제) ③ project 별 last_learned_at 미구현.

G7. 다중 priority 합성 (§5.9) — 부분

현재 diameter/longest/utility 단일 키. 사양의 $0.35 \cdot \text{관경} + 0.25 \cdot \text{길이} + 0.20 \cdot \text{PoC 혼잡도} + 0.15 \cdot \text{번들중심성} + 0.05 \cdot \text{유틸우선}$ 합성 score 없음(특히 poc_congestion·bundle centrality).

G8. feature 시각 검증 레이어 (§9.3) — 부분

학습된 rack level / bundle corridor / stub / face 를 3D 오버레이로 보는 레이어가 부분적.
기존/자동 동일 색 비교 overlay 는 일부 있음.

G9. 모듈 분리 (§11) — 부분(리팩터링)

로직이 SceneViewModel(거대)·DbRouteDiag·pattern_learn 에 산재. 사양은
RouteFeatureExtractor/RoutePatternLearner/RouteFeatureStore/RouteProfileResolver/AutoRoutePlanner/RouteValidator 분리 권장.

3. 개발 계획 (4 페이지즈)

사양 §10 우선순위를 현 구현 현실에 맞게 재배열. 각 페이지즈는 골든 불변·기존 동작 보존(옵트인)을 지킨다.

Phase F1 — 특징점 저장소 + 학습 파이프라인 정식화 (G2·G4·G6)

"흠어진 학습 산출물을 `route_feature_*` 로 통합하고 batch/증분 학습을 만든다." 자동설계 동작은 그대로 두고 **저장소만 먼저 구축**(위험 낮음, 이후 모든 기능의 토대).

- F1-1. `db/schema/route_feature.sql` — 6 테이블 + 인덱스 DDL(사양 §6.2 그대로) `-- apply-schema`.
- F1-2. `RouteFeatureExtractor`(신규, C# 또는 Python) — `route_path` 1 건 → `route_feature_path`(형상)+`route_feature_anchor`(접속면/rel_pos)+stub template. 기존 `pattern_learn`·`StubExtractor`·`bundle_detect` 로직 재사용.
- F1-3. `route_feature_group_profile` 머티리얼라이저 — 집계 + fallback 키 5 단계 upsert.
- F1-4. 증분 학습: `route_hash`(guid+points+util+equip+target) + project `last_learned_at` + `is_training_usable` 품질게이트(이상 경로 배제).
- F1-5. CLI `pattern_learn --feature {--write-db|--incremental}` 확장(또는 신규 `feature_learn`).

Phase F2 — group profile resolver + 자동설계 통합 적용 (G3·G7)

"라우팅 시작 시 profile 한 번 읽어 face/rack/corridor/pitch/radius/priority 를 일괄 구성."

- F2-1. `RouteProfileResolver`(신규) — (project,equip,util,grp,diameter) → `route_feature_group_profile` exact→fallback 조회. `PatternStore`/`BundleStore` 를 흡수/위임.
- F2-2. `SceneViewModel.BuildEngineForRows` 가 resolver 결과로 face 투영·rack_levels·corridor·per-task radius-gap 를 **일괄** 세팅(현 산발 호출 정리).
- F2-3. 합성 priority score(§5.9) — 엔진 `RouteTask` 에 congestion/centrality 입력 또는 C# 선정렬. C ABI `r3d_set_task_diameter` 패턴 따라 확장.
- F2-4. 헤드리스 A/B(`--dbroute`)에 profile-on/off env 추가, 회귀(성공률/유사도) 측정.

Phase F3 — 장애물-배관 연관 학습 + 엔진 3 계층 비용 (G1) — 가장 큰 신규

"사람처럼 미리 꺾고 띄우고 복귀." 학습 + 엔진 비용 양쪽 신규.

- F3-1. `ObstacleClassifier` — AABB 중형비/OST_TYPE/DDWORKS_TYPE 로 COLUMN/H_BEAM/WALL/FRAME 분류 + `obstacle_axis`.

- F3-2. **ObstacleRelationExtractor** — 최근접 거리·확장 AABB 교차·전후 bend 거리·bypass side/axis-z_delta-extra_length + **relation_score**(5 성분) → **route_feature_obstacle_relation**.
- F3-3. preferred bypass profile
집계((obstacle_type,axis,util_group,util)→side/clearance/bend_before/after/z_delta).
- F3-4. **엔진 비용 3 계층**(신규, 골든 불변·옵트인):
• hard zone(기존 장애물)
• **soft clearance zone** — 환경+이격 팽창 영역에 가산 비용. C ABI
r3d_set_soft_clearance(...) 또는 셀 집합 주입.
- **preferred bypass corridor** — 학습 우회 side 셀에 감산(기존 corridor 인프라 **r3d_set_corridor_cells** 재사용 가능성 검토).
- F3-5. waypoint 후보(장애물 전 **bend_before_mm**)를 corridor/goal_dir 로 주입.
- F3-6. ctest(소격자에서 soft/preferred OFF=골든 바이트 불변) + 헤드리스 A/B(우회 side 일치율).

Phase F4 — 유사도 검증·리포트·시각화 (G5·G8·G9)

- F4-1. **RouteValidator**(신규) — per-task 7 성분 similarity(\$5.10) + 작업별/그룹별 리포트(\$9.1·9.2:
face_match·corridor_overlap·collision_count·rack_z_match·bundle_match).
- F4-2. **AutoDesignReport** 확장 — 기존설계 대비 per-task 비교 표 + similarity 컬럼.
- F4-3. 3D feature 레이어 토글 — 학습 rack level(수평 평면)·bundle corridor(셀)·stub(고정구간)·face(화살표) overlay. 기존/자동 동일색 비교.
- F4-4. (선택) §11 모듈 리팩터링 — **SceneViewModel** 거대 메서드를 위 신규 모듈로 추출.

4. 개발 리스트 (체크리스트)

우선순위 P1(토대)→P4(검증). 각 항목 옆 [모듈] = 작업 위치, (난이도 S/M/L).

DB / 학습 (Phase F1·F3 학습부)

- [] **db/schema/route_feature.sql** 6 테이블+인덱스 DDL [db] (S)
- [] **route_feature_path** 추출(형상:
total/manhattan/detour/bend/vertical_ratio/main_rack_z/dominant_axis/bbox/normalized_points) [신규 Extractor] (M)

- [] `route_feature_anchor` 추출(face-direction_unit-rel_pos-rise-confidence) — `pattern_learn` 재사용 [Extractor] (M)
- [] `route_feature_stub_template` medoid 대표 스텝 — 현 평균/mode 를 medoid 로 보강 [pattern_learn] (M)
- [] `route_feature_bundle_template`(trunk_zs·trunk_centerline·pitch·lane-shared_ratio·members) — `bundle_detect` 확장 [bundle_detect] (M)
- [] `route_feature_obstacle_relation` 추출 — 장애물 분류+우회 profile [신규 ObstacleRelationExtractor] (L)
- [] `route_feature_group_profile` 집계/머티리얼라이즈 + fallback 5 단계 upsert [신규] (M)
- [] 증분 학습(route_hash·last_learned_at) [pattern pipeline] (M)
- [] 품질 게이트(`is_training_usable`: PoC 거리/순서/길이/꺾임/anchor/관통 이상 배제) [Extractor] (M)
- [] 학습 CLI(`feature_learn --write-db|--incremental|--report`) [신규] (S)

자동설계 적용 (Phase F2·F3 적용부)

- [] `RouteProfileResolver`(exact→fallback) [신규] (M)
- [] `BuildEngineForRows` 가 resolver 로 face/rack/corridor/radius/gap 일괄 세팅 [SceneViewModel] (M)
- [] 합성 priority score(관경+길이+PoC 혼잡도+번들중심성) [multi_route/SceneViewModel] (M)
- [] 엔진 **soft clearance zone 비용** (C ABI + astar 비용, 옵트인·골든 불변) [cpp/capi] (L)
- [] 엔진 **preferred bypass corridor 비용** (기존 corridor 인프라 재사용 검토) [cpp/capi] (M)
- [] 장애물 전방 bend waypoint 후보 주입 [SceneViewModel] (M)
- [] H-beam 상/하/측면 우회 비용 차등(beam axis+높이) [cpp/capi+C#] (L)
- [] 스텝 우선 라우팅을 profile-stub 기준으로 일원화(현 StubExtractor 직접 → template 우선) [SceneViewModel] (M)

검증 / 시각화 (Phase F4)

- [] `RouteValidator` per-task 7 성분 similarity [신규] (M)
- [] AutoDesignReport 에 기존 대비 per-task 비교(face_match/corridor_overlap/collision) [AutoDesignReport] (M)
- [] 그룹별 리포트(success_rate/grouping_factor/rack_z_match/bundle_match) [AutoDesignReport] (S)

- [] 3D feature 레이어(rack/corridor/stub/face overlay) 토글 [SceneViewModel+MainWindow] (M)
- [] 실패 사유 자동 분류(\$7.8 7 범주) — ExplainFailure 확장 [SceneViewModel] (S)
- [] (선택) 설계자 승인/수정 feedback 저장 → 재학습 [신규] (L)

리팩터링 (Phase F4 선택)

- [] \$11 모듈 분리(Extractor/Learner/Store/Resolver/Planner/Validator) [전반] (L)
- [] Python 레퍼런스 DDW_AI_DB 재작성(현 AUTOROUTINGV7 기반 obstacle_db/scene/route_db) [python] (M)

5. 권장 결정사항 (착수 전 확인 필요)

1. 저장소 통합 범위 — 사양의 route_feature_* 6 테이블로 완전 통합할지, 아니면 기존 route_stub_pattern/route_bundle_group 을 유지하고 부족분만 추가(obstacle_relation_group_profile)할지. → 권장: 기존 유지 + 부족분 추가(마이그레이션 비용 ↓, 회귀 위험 ↓).
2. 학습 주체 언어 — feature 추출을 Python(pattern_learn 확장, pgvector/통계 강점)에서 할지, C#(DbRouteDiag/뷰어가 실행 주체)에서 할지. → 권장: Python batch 학습 + C# 런타임 resolver(현 구조와 동일, L1~L4 선례).
3. 장애물 분류 소스 — DDW_AI_DB 의 OST_TYPE/DDWORKS_TYPE/COLLISION_PASS 만으로 COLUMN/H_BEAM 구분이 가능한지 실데이터 확인 필요(불가하면 AABB 중첩비 휴리스틱 병행).
4. 엔진 비용 확장 범위 — soft clearance/preferred bypass 를 엔진(C++) 비용함수에 넣을지, C# corridor 셀 주입으로 근사할지. → 권장: preferred bypass 는 기존 r3d_set_corridor_cells 재사용, soft clearance 만 엔진 신규(골든 불변 옵트인).
5. 우선순위 — 가장 가치 큰 Phase F3(장애물-배관 연관)을 먼저 할지, 토대인 F1(저장소)→F2(resolver) 를 먼저 깔지. → 권장: F1 최소(obstacle_relation 포함 DDL+추출) → F3(엔진 비용) 우선, F2/F4 병행.

6. 결론

자동설계 적용 계층(②)은 제품에 근접(스텝·rack·번들·corridor·관경·rip-up·복제·비교리포트
완비)하나, 사양이 요구하는 (a) 정식 feature 저장소·group profile resolver 와 (b) 장애물-배관
연관 학습 + 엔진 3 계층(hard/soft/preferred) 비용이 핵심 미구현이다.

가장 큰 제품 가치는 **G1(장애물-배관 연관)** — "기둥/H-beam 앞에서 사람처럼 미리 꺾고
띄우고 복귀"를 만들어 현재 미해결인 *혼잡부 계단·우회 품질*(P3t 한계)을 근본 개선한다. 이를
위해 F1(저장소 토대)→F3(연관 학습+엔진 비용)을 우선 축으로, F2(resolver)·F4(검증)를
병행하는 것을 권장한다.